

Sec. 6.3. Hashing Basics

Hashing Basics

Task 1 - Introduction

A **hash value** is a fixed-size string or characters that is computed by a hash function. A **hash function** takes an input of an arbitrary size and returns an output of fixed length, i.e., a hash value.

Task 2 - Hash Functions

hexdump A hexdump is a representation of binary data where each byte is shown as a hexadecimal value, often alongside its ASCII interpretation.

Consequently, the following two files differ by a single bit. However, if we compare their MD5 (Message-Digest Algorithm 5) hashes, their SHA1 (Secure Hash Algorithm 1) hashes, or their SHA-256 (Secure Hash Algorithm 256) hashes, we will notice that they are entirely different. We recommend that you try the commands below yourself. The files are located in `~/Hashing-Basics/Task-2/`.

```
strategos@g5000 ~-> hexdump -C file1.txt
00000000  54                                     |T|
00000001
strategos@g5000 ~-> md5sum *.txt
b9ece18c950afbfa6b0fdbfa4ff731d3  file1.txt
4c614360da93c0a041b22e537de151eb  file2.txt
strategos@g5000 ~-> sha1sum *.txt
c2c53d66948214258a26ca9ca845d7ac0c17f8e7  file1.txt
b2c7c0caa10a0cca5ea7d69e54018ae0c0389dd6  file2.txt
strategos@g5000 ~-> sha256sum *.txt
e632b7095b0bf32c260fa4c539e9fd7b852d0de454e9be26f24d0d6f91d069d3  file1.txt
a25513c7e0f6eaa80a3337ee18081b9e2ed09e00af8531c8f7bb2542764027e7  file2.txt
```

The output of a hash function is typically raw bytes, which are then encoded. Common encodings are base64 or hexadecimal. `md5sum`, `sha1sum`, `sha256sum`, and `sha512sum` produce their outputs in hexadecimal format.

Why is Hashing Important? Hashing helps protect data's integrity and ensure password confidentiality.

What's a Hash Collision? A hash collision is when two different inputs give the same output. Hash functions are designed to avoid collisions as best as possible.

Answer the questions below

What is the SHA256 hash of the passport.jpg file in `~/Hashing-Basics/Task-2?`

77148c6f605a8df855f2b764bcc3be749d7db814f5f79134d2aa539a64b61f02

```
user@ip-10-114-154-231:~/Hashing-Basics/Task-2$ ls
file1.txt  file2.txt  passport.jpg
user@ip-10-114-154-231:~/Hashing-Basics/Task-2$ sha256sum passport.jpg
77148c6f605a8df855f2b764bcc3be749d7db814f5f79134d2aa539a64b61f02  passport.jpg
user@ip-10-114-154-231:~/Hashing-Basics/Task-2$ md5sum passport.jpg
5c58ca24e8f64c058e2eda9928fd0f97  passport.jpg
user@ip-10-114-154-231:~/Hashing-Basics/Task-2$
```

What is the output size in bytes of the MD5 hash function? 16

If you have an 8-bit hash output, how many possible hash values are there? 256

Task 3 - Insecure Password Storage for Authentication

Storing Passwords in Plaintext: They stored their passwords in **plaintext**, and the company had a data breach. The text file contains over 14 million passwords. You can find `rockyou.txt` in the `/usr/share/wordlists` directory.

```
strategos@g5000 /usr/share/wordlists> wc -l rockyou.txt
14344392 rockyou.txt
strategos@g5000 /usr/share/wordlists> head rockyou.txt
123456
12345
123456789
password
```

Using an Insecure Encryption Algorithm: the plaintext password could be retrieved relatively quickly.

Using an Insecure Hash Function: LinkedIn used an insecure hashing algorithm, the SHA-1, to store user passwords. Furthermore, no password salting was used. **Password salting** refers to adding a **salt**, i.e., a random value, to the password before it is hashed.

Answer the questions below

| What is the 20th password in rockyou.txt? qwerty

```
user@ip-10-114-154-231:~$ head -n20 /usr/share/wordlists/rockyou.txt
123456
12345
123456789
password
iloveyou
princess
1234567
rockyou
12345678
abc123
nicole
daniel
babygirl
monkey
lovely
jessica
654321
michael
ashley
qwerty
```

Task 4 - Using Hashing for Secure Password Storage

Websites like [CrackStation\(opens in new tab\)](#) and [Hashes.com\(opens in new tab\)](#), internally use massive rainbow tables to provide fast password cracking for **hashes without salts**.

[Defuse.ca](#) · [Twitter](#)

[CrackStation](#) · [Password Hashing Security](#) · [Defuse Security](#)

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

c44a471bd78cc6c2fea32b9fe028d30a

I'm not a robot

reCAPTCHA

Privacy · Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, rpeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Hash	Type	Result
c44a471bd78cc6c2fea32b9fe028d30a	md5	asdfghjkl

Color Codes: Exact match, Partial match, Not found.

Download CrackStation's Wordlist

How CrackStation Works

CrackStation uses massive pre-computed lookup tables to crack password hashes. These tables store a mapping between the hash of a password, and the correct password for that hash. The hash values are indexed so that it is possible to quickly search the database for a given hash. If the hash is present in the database, the password can be recovered in a fraction of a second. This only works for "unsalted" hashes. For information on password hashing systems that are not vulnerable to pre-computed lookup tables, see our [hashing security page](#).

CrackStation's lookup tables were created by extracting every word from the Wikipedia databases and adding with every password list we could find. We also applied intelligent word mangling (brute force hybrid) to our wordlists to make them much more effective. For MD5 and SHA1 hashes, we have a 190GB, 15-billion-entry lookup table, and for other hashes, we have a 19GB 1.5-billion-entry lookup table.

You can download CrackStation's dictionaries [here](#), and the lookup table implementation (PHP and C) is available [here](#).

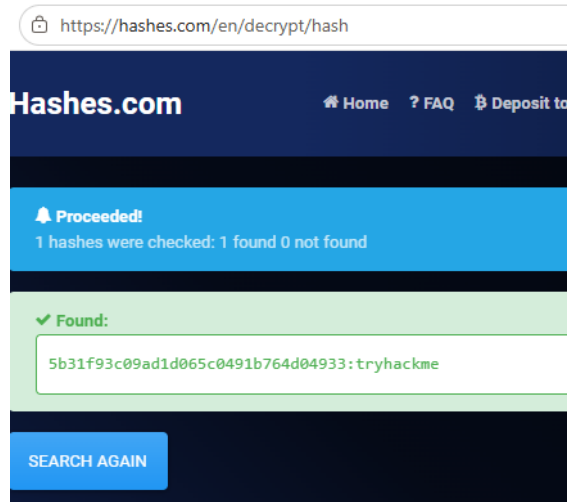
To protect against rainbow tables, we add a salt to the passwords. The salt is a randomly generated value stored in the database and should be unique to each user. In theory, you could use the same salt for all users, but duplicate passwords would still have the same hash and a rainbow table could still be created for passwords with that salt.

The salt is added to either the start or the end of the password before it's hashed, and this means that every user will have a different password hash even if they have the same password. Hash functions like Bcrypt and Scrypt handle this automatically. Salts don't need to be kept private.

Hash	Password
02c75fb22c75b23dc963c7eb91a062cc	zxcvbnm
b0baee9d279d34fa1dfd71aadb908c3f	11111
c44a471bd78cc6c2fea32b9fe028d30a	asdfghjkl
d0199f51d2728db6011945145a1b607a	basketball
dcddb75469b4b4875094e14561e573d8	000000
e10adc3949ba59abbe56e057f20f883e	123456
e19d5cd5af0378da05f63f891c7467af	abcd1234
e99a18c428cb38d5f260853678922e03	abc123
fcea920f7412b5da7be0cf42b8c93759	1234567
4c5923b6a6fac7b7355f53bfe2b8f8c1	inS3CyourP4\$\$

Answer the questions below

- Manually check the hash "4c5923b6a6fac7b7355f53bfe2b8f8c1" using the rainbow table above. inS3CyourP4\$\$
- Crack the hash "5b31f93c09ad1d065c0491b764d04933" using an online tool. tryhackme



| Should you encrypt passwords in password-verification systems? Nay

Task 5 - Recognising Password Hashes

On Linux, password hashes are stored in `/etc/shadow`, which is normally only readable by root. They used to be stored in `/etc/passwd`, which was readable by everyone.

Prefix	Algorithm
<code>\$y\$</code>	yescrypt is a scalable hashing scheme and is the default and recommended choice in new systems
<code>\$gy\$</code>	gost-yescrypt uses the GOST R 34.11-2012 hash function and the yescrypt hashing method
<code>\$7\$</code>	scrypt is a password-based key derivation function
<code>\$2b\$</code> , <code>\$2y\$</code> , <code>\$2a\$</code> , <code>\$2x\$</code>	bcrypt is a hash based on the Blowfish block cipher originally developed for OpenBSD but supported on a recent version of FreeBSD, NetBSD, Solaris 10 and newer, and several Linux distributions
<code>\$6\$</code>	sha512crypt is a hash based on SHA-2 with 512-bit output originally developed for GNU libc and commonly used on (older) Linux systems
<code>\$md5</code>	SunMD5 is a hash based on the MD5 algorithm originally developed for Solaris
<code>\$1\$</code>	md5crypt is a hash based on the MD5 algorithm originally developed for FreeBSD

```
root@TryHackMe# sudo cat /etc/shadow | grep strategos
strategos:$y$j9T$76UzfgEM5PnymhQ7TLJey1$/00Sg64dhfF.TigVPdzqiFang6uZA4QA1pzzegKdVm4:19965:0:9999:7:::
```

The fields are separated by colons. The important ones are the username and the hash algorithm, salt, and hash value. The second field has the format `$prefix$options$salt$hash`.

In the example above, we have four parts separated by `$`:

- `y` indicates the hash algorithm used, **yescrypt**
- `j9T` is a parameter passed to the algorithm
- `76UzfgEM5PnymhQ7TLJey1` is the salt used

- `/00Sg64dhfF.TigVPdzqiFang6uZA4QA1pzzeKdVm4` is the hash value

MS Windows Passwords

A great place to find more hash formats and password prefixes is the [Hashcat Example Hashes\(opens in new tab\)](#) page. For other hash types, you'll typically need to check the length or encoding or even conduct some research into the application that generated them. Never underestimate the power of research.

Answer the questions below

- | What is the hash size in yescript? 256 (Researched online)
- | What's the Hash-Mode listed for Cisco-ASA MD5? 2410 (Checked the Hashcat Example Hashes page)
- | What hashing algorithm is used in Cisco-IOS if it starts with 9? scrypt

Task 6 - Password Cracking

Once it matches, you know what the password was. Tools like [Hashcat\(opens in new tab\)](#) and [John the Ripper\(opens in new tab\)](#) are commonly used for these purposes.

Cracking Passwords with GPUs: Some hashing algorithms, such as Bcrypt, are designed so that hashing on a GPU does not provide any speed improvement over using a CPU; this helps them resist cracking.

Cracking on VMs? If you want to run [Hashcat\(opens in new tab\)](#), it's best to run it on your host to make the most of your GPU, if available. [John the Ripper\(opens in new tab\)](#) uses CPU by default and works in a VM out of the box, although you may get better speeds running it on the host OS to avoid any virtualisation overhead and make the most of your CPU cores and threads.

Time to Crack Some Hashes: [Hashcat\(opens in new tab\)](#), or [John the Ripper\(opens in new tab\)](#). Although you can use [online rainbow tables\(opens in new tab\)](#) to solve the following, we strongly advise against doing that as this will restrict your learning experience. For the first three questions, using `hashcat` along with `rockyou.txt` is enough to find the answers.

Hashcat is a password-cracking and recovery tool commonly used by security professionals to test the strength of passwords. It works by comparing password guesses against stored hashes and supports various attack methods such as dictionary and brute-force attacks.

For example, `hashcat -m 3200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt` will treat the hash as Bcrypt and try the passwords in the `rockyou.txt` file. This command uses Hashcat to perform a dictionary attack against hashes stored in `hash.txt` using the `rockyou.txt` wordlist.

hashcat Starts the Hashcat tool
-m 3200 Specifies the hash type. 3200 corresponds to bcrypt
-a 0 Attack mode 0, which is a straight/dictionary attack
hash.txt File containing the target hash(es)
/usr/share/wordlists/rockyou.txt Wordlist containing passwords to test

What Happens? Reads the bcrypt hash(es) from `hash.txt`.

Takes each password candidate from `rockyou.txt`.

Hashes each candidate using bcrypt.

Compares the generated hash to the target hash.

If a match is found, the original password is identified.

"This Hashcat command performs a dictionary attack against bcrypt hashes. The `-m 3200` option tells Hashcat the hashes are bcrypt, and `-a 0` specifies a straight dictionary attack. Hashcat reads passwords from the RockYou wordlist and compares their bcrypt hashes to those stored in `hash.txt` until a match is found."

Answer the questions below:

- | Create **vim hash1.txt** then run **hashid hash1.txt** get SHA512 Crypt , checked id from table given 1800 then run command `hashcat -m 1800 -a 0 hash1.txt /usr/share/wordlists/rockyou.txt`

Use hashcat to crack the hash, \$2a\$06\$7yoU3Ng8dHTXphAg913cyO6Bjs3K5lBnwq5FJyA6d01pMSrddr1ZG, saved in ~/Hashing-Basics/Task-6/hash1.txt. 85208520

Use hashcat to crack the SHA2-256 hash, 9eb7ee7f551d2f0ac684981bd1f1e2fa4a37590199636753efe614d4db30e8e1, saved in saved in ~/Hashing-Basics/Task-6/hash2.txt. halloween

Use hashcat to crack the hash, \$6\$GQXVvW4EuM\$ehD6jWiMsfNorxy5SINsgdIxmAEI3.yif0/c3NqzGLa0P.S7KRdYjycw5bnYkF5ZtB8wQy8KnskuWQS3Yr saved in ~/Hashing-Basics/Task-6/hash3.txt. spaceman

Crack the hash, b6b0d451bbf6fed658659a9e7e5598fe, saved in ~/Hashing-Basics/Task-6/hash4.txt. funforyou

Task 7 - Hashing for Integrity Checking

The text file listed below shows the SHA256 hash of two Fedora Workstation ISO files. If running `sha256sum` on the file you downloaded returned the same hash listed in this signed file, you can be confident that your file is identical to the official one.

```
root@AttackBox# head Fedora-Workstation-40-1.14-x86_64-CHECKSUM
-----BEGIN PGP SIGNED MESSAGE-----
Hash: SHA256# Fedora-Workstation-Live-osb-40-1.14.x86_64.iso: 2623733760 bytes
SHA256 (Fedora-Workstation-Live-osb-40-1.14.x86_64.iso) = 8d3cb4d99f27eb932064915bc9ad34a7529
d5d073a390896152a8a899518573f# Fedora-Workstation-Live-x86_64-40-1.14.iso: 2295853056 bytes
SHA256 (Fedora-Workstation-Live-x86_64-40-1.14.iso) = dd1faca950d1a8c3d169adf2df4c3644ebb62f8
aac04c401f2393e521395d613
[...]
```

HMAC (Keyed-Hash Message Authentication Code) is a type of message authentication code (MAC) that uses a cryptographic hash function in combination with a secret key to verify the authenticity and integrity of data.

An HMAC can be used to ensure that the person who created the HMAC is who they say they are, i.e., authenticity is confirmed; moreover, it proves that the message hasn't been modified or corrupted, i.e., integrity is maintained. This is achieved through the use of a secret key to prove authenticity and a hashing algorithm to produce a hash and prove integrity. The following steps give you a fair idea of how HMAC works.

1. The secret key is padded to the block size of the hash function.
2. The padded key is XORed with a constant (usually a block of zeros or ones).
3. The message is hashed using the hash function with the XORed key.
4. The result from Step 3 is then hashed again with the same hash function but using the padded key XORed with another constant.
5. The final output is the HMAC value, typically a fixed-size string.

The illustration below should clarify the above steps.

Answer the questions below

What is SHA256 hash of libgcrypt-1.11.0.tar.bz2 found in ~/Hashing-Basics/Task-7?
09120c9867ce7f2081d6aaa1775386b98c2f2f246135761aae47d81f58685b9c

```
user@ip-10-114-177-5:~/Hashing-Basics/Task-7$ sha256sum libgcrypt-1.11.0.tar.bz2
09120c9867ce7f2081d6aaa1775386b98c2f2f246135761aae47d81f58685b9c libgcrypt-1.11.0.ta
r.bz2
user@ip-10-114-177-5:~/Hashing-Basics/Task-7$
```

What's the hashcat mode number for HMAC-SHA512 (key = \$pass)? 1750

Task 8 - Conclusion

Hashing, as already stated, is a process that takes input data and produces a hash value, a fixed-size string of characters, also referred to as digest. This hash value uniquely represents the data, and any change in the data, no matter how small, should lead to a change in the hash value. Hashing should not be confused with encryption or encoding; hashing is one-way, and you can't reverse the process to get the original data.

Encoding converts data from one form to another to make it compatible with a specific system. ASCII, UTF-8, UTF-16, UTF-32, ISO-8859-1, and Windows-1252 are valid encoding methods for the English language. Note that UTF-8, UTF-16, and UTF-32 are Unicode encodings, and they can represent characters from other languages, such as Arabic and Japanese. Another type of encoding commonly used when sending or saving data is not for any specific language. Examples include Base32 and Base64 encoding. Consider the following example of using `base64` to encode and decode.

```
strategos@g5000 ~-> base64
TryHackMe
VHJ5SGFja01lCg==
strategos@g5000 ~-> base64 -d
VHJ5SGFja01lCg==
TryHackMe
```

Encoding should not be confused with encryption, as using a specific encoding does not protect the confidentiality of the message. Encoding is reversible; anyone can change the data encoding with the right tools.

Encryption, protects data confidentiality using a cryptographic cipher and a key. Encryption is reversible, provided we know the cipher and can access the key.

Answer the questions below

Use base64 to decode `RU5jb2RlREVjb2RlCg==`, saved as `decode-this.txt` in `~/Hashing-Basics/Task-8`. What is the original word? `ENcodeDEcode` (or `echo "RU5jb2RlREVjb2RlCg==" | base64 -d`)

```
user@ip-10-114-177-5:~/Hashing-Basics/Task-7$ cd ../Task-8
user@ip-10-114-177-5:~/Hashing-Basics/Task-8$ cat decode-this.txt |base64 -d
ENcodeDEcode
user@ip-10-114-177-5:~/Hashing-Basics/Task-8$ █
```